Metody Inteligencji Obliczeniowej

Sprawozdanie 3

Wydział Fizyki i Informatyki Stosowanej

Mikhail Shupliakou

05.04.2026

Spis treści

1 Wstęp **3**

2 Zadanie 1 **3**
2.1 Realizacja zadania 3
2.2 Wyniki i wnioski 4

3 Zadanie 2 **5**
3.1 Realizacja zadania 5
3.2 Wyniki i wnioski 6

4 Zadanie 3 **8**
4.1 Realizacja zadania 8
4.2 Wyniki i wnioski 9

Aproksymacja funkcji

1 Wstęp

Aproksymacja funkcji na podstawie danych empirycznych stanowi jedno z najważniejszych zagadnień we współczesnej analizie danych i uczeniu maszynowym. Pozwala ona na odnajdywanie i uogólnianie ukrytych zależności pomiędzy zmiennymi wejściowymi a pożądanymi wartościami wyjściowymi, nawet w przypadku zjawisk o wysoce nieliniowym charakterze.

W niniejszym sprawozdaniu przedstawiono proces budowy i ewaluacji modeli regresyjnych opartych na sztucznych sieciach neuronowych (ang. *Artificial Neural Networks*), których zadaniem jest aproksymacja badanej funkcji ciągłej. Głównym celem zrealizowanych eksperymentów jest zbadanie wpływu doboru kluczowych hiperparametrów sieci na jakość dopasowania modelu. W szczególności przeanalizowano różne architektury (liczbę warstw ukrytych oraz liczbę neuronów), a także zbadano efektywność popularnych funkcji aktywacji, takich jak ReLU oraz tangens hiperboliczny (*tanh*).

Aby rzetelnie ocenić zdolność modeli do generalizacji, zbiór danych poddano procesowi standaryzacji, a następnie podzielono na podzbiór uczący i testujący. Jako główne kryterium oceny błędu aproksymacji przyjęto błąd średniokwadratowy (ang. *Mean Squared Error* – MSE). Przeprowadzona analiza porównawcza pozwala na wyłonienie optymalnej struktury sieci, zapewniającej największą dokładność predykcji.

2 Zadanie 1

Celem niniejszego zadania jest aproksymacja funkcji modelującej wpływ wydatków reklamowych na wielkość sprzedaży z wykorzystaniem sztucznych sieci neuronowych typu MLP (ang. *Multilayer Perceptron*). Istota pracy polega na odpowiednim przygotowaniu danych, a następnie na budowie i empirycznym porównaniu modeli o różnych architekturach warstw ukrytych i funkcjach aktywacji (ReLU oraz tanh). Ewaluacja skuteczności poszczególnych wariantów opiera się na wyznaczeniu błędu średniokwadratowego (MSE) na odseparowanym zbiorze testowym oraz na wizualnej analizie predykcji, co pozwala na wyłonienie konfiguracji o najwyższych zdolnościach uogólniania.

2.1 Realizacja zadania

W ramach realizacji zadania przeprowadzono proces aproksymacji funkcji przewidującej wielkość sprzedaży na podstawie wydatków na reklamę (TV, radio, prasa). Na wstępnym etapie zbiór danych został losowo podzielony na próbę uczącą (80%) i testową (20%), przy zachowaniu stałego ziarna losowości (*random_state=42*) w celu zapewnienia powtarzalności eksperymentów. Następnie, aby algorytmy optymalizacyjne działały prawidłowo, dokonano standaryzacji cech wejściowych (Z-score normalization) z wykorzystaniem klasy `StandardScaler`. Parametry skalowania zostały wyznaczone wyłącznie na zbiorze treningowym, a następnie zaaplikowane do obu podzbiorów, co pozwoliło uniknąć zjawiska wycieku danych (ang. *data leakage*).

Główny etap badań polegał na zbudowaniu modeli regresyjnych opartych na sieciach neuronowych typu Multilayer Perceptron (`MLPRegressor`). Zgodnie z wymaganiami, zaimplementowano pętlę iteracyjną, która systematycznie trenowała i ewaluowała modele dla różnych kombinacji hiperparametrów.

- **Struktura warstw ukrytych:** Zbadano dwie różne architektury – płytszą sieć składającą się z dwóch warstw po 50 neuronów `(50, 50)` oraz sieć głęboką o pięciu warstwach po 100 neuronów `(100, 100, 100, 100, 100)`.
- **Funkcje aktywacji:** Dla każdej z architektur sprawdzono działanie dwóch nieliniowych funkcji aktywacji: ReLU (`relu`) oraz tangensa hiperbolicznego (`tanh`).

Do optymalizacji wag użyto algorytmu Adam (`solver='adam'`). Aby zapewnić zbieżność procesu uczenia, maksymalną liczbę epok ustalono na 5000, a tolerancję błędu na poziomie 0.001. Jakość aproksymacji dla każdego wariantu sieci była na bieżąco weryfikowana na odseparowanym zbiorze testowym za pomocą błędu średniokwadratowego (MSE).

Kluczowy fragment kodu, realizujący ten etap prac, przedstawiono na poniższym listingu:

```
architectures = [(50, 50), (100, 100, 100, 100, 100)]
activations = ['relu', 'tanh']

for arch in architectures:
    for act in activations:
        network = MLPRegressor(
            solver='adam',
            hidden_layer_sizes=arch,
            max_iter=5000,
            tol=0.001,
            activation=act,
            random_state=42
        )

        network.fit(X_train_scaled, y_train)

        y_pred = network.predict(X_test_scaled)
        mse = mean_squared_error(y_test, y_pred)
```

2.2 Wyniki i wnioski

Po przeprowadzeniu procesu uczenia i ewaluacji modeli na zbiorze testowym, otrzymano wyniki błędu średniokwadratowego (MSE) dla każdej z badanych konfiguracji. Zestawienie uzyskanych wartości prezentuje Tabela 1.

Struktura warstw ukrytych	Funkcja aktywacji	Błąd MSE
(50, 50)	ReLU	0.6508
(50, 50)	tanh	0.6820
(100, 100, 100, 100, 100)	ReLU	0.5681
(100, 100, 100, 100, 100)	tanh	**0.2988**

Tabela 1: Zestawienie błędu średniokwadratowego (MSE) dla badanych modeli

Na podstawie analizy powyższych wyników można sformułować następujące wnioski:

1. **Wpływ architektury (głębokości sieci):** Zwiększenie złożoności modelu poprzez dodanie kolejnych warstw ukrytych i neuronów pozytywnie wpłynęło na jego zdolności aproksymacyjne. Głęboka sieć o strukturze `(100, 100, 100, 100, 100)` osiągnęła zauważalnie niższe wartości błędu MSE w porównaniu do płytszej sieci `(50, 50)`, niezależnie od zastosowanej funkcji aktywacji.

2. **Wpływ funkcji aktywacji:** Skuteczność funkcji aktywacji okazała się silnie uzależniona od architektury sieci. W przypadku modelu płytkiego `(50, 50)`, obie funkcje dały zbliżone rezultaty, z niewielką przewagą funkcji ReLU (0.6508 vs 0.6820). Jednakże, w przypadku modelu głębokiego, zastosowanie tangensa hiperbolicznego (*tanh*) doprowadziło do drastycznej poprawy wyników – błąd zmalał niemal o połowę w stosunku do wariantu z funkcją ReLU (0.2988 vs 0.5681).

3. **Optymalny model:** Najlepszą konfiguracją do rozwiązania postawionego problemu aproksymacji sprzedaży okazała się pięciowarstwowa sieć neuronowa `(100, 100, 100, 100, 100)` wykorzystująca funkcję aktywacji `tanh`. Osiągnęła ona najniższy błąd średniokwadratowy na poziomie 0.2988, co świadczy o jej najwyższej dokładności i najlepszych zdolnościach do generalizacji wiedzy na nowych, niewidzianych wcześniej danych.

Podsumowując, eksperyment dowodzi, że do zamodelowania nieliniowych zależności między wydatkami na różne kanały reklamowe a końcową sprzedażą, bardziej złożone i głębokie architektury sieci neuronowych w połączeniu z odpowiednio dobraną funkcją aktywacji (w tym przypadku *tanh*) przynoszą optymalne rezultaty.

3 Zadanie 2

Celem drugiego zadania jest budowa modeli sieci neuronowych do przewidywania postępu cukrzycy (problem regresji) na podstawie zbioru *diabetes* ze *scikit-learn*. Praca obejmuje standaryzację danych, podział na zbiory uczący i testujący oraz ewaluację co najmniej pięciu różnych architektur sieci. Jakość aproksymacji oceniono za pomocą wskaźników MSE, MAPE oraz R^2. Ostatnim etapem jest identyfikacja i wizualizacja predykcji najlepszego z modeli, co pozwala wyłonić najmniejszą architekturę gwarantującą zadowalające rezultaty.

3.1 Realizacja zadania

W ramach drugiego zadania przeprowadzono proces uczenia modeli regresyjnych na rzeczywistym zbiorze danych *diabetes*, udostępnianym w pakiecie *scikit-learn*. Zbiór ten zawiera 10 bazowych cech medycznych pacjentów (m.in. wiek, płeć, wskaźnik BMI, ciśnienie krwi) oraz docelową liczbową miarę postępu choroby po roku. Podobnie jak w poprzednim eksperymencie, dane podzielono na podzbiór uczący (80%) i testujący (20%) z zachowaniem ziarna losowości (*random_state=42*), a następnie poddano standaryzacji (*StandardScaler*) wyliczonej na zbiorze treningowym.

Głównym etapem była implementacja procedury testującej pięć różnych architektur sieci neuronowych (MLP) o rosnącej złożoności: od prostej sieci z jedną płytką warstwą ukrytą (10 neuronów), po bardziej rozbudowane, głębokie struktury (np. 3 warstwy po 100 neuronów). Ze względu na to, że celem jest badanie wpływu architektury, dla wszystkich modeli zachowano stałą funkcję aktywacji (domyślne ReLU) oraz solver (Adam).

Dla każdego testowanego wariantu obliczono trzy metryki ewaluacyjne na odseparowanym zbiorze testowym: błąd średniokwadratowy (MSE), średni absolutny błąd procentowy (MAPE) oraz współczynnik determinacji (R^2). Skrypt został zaprojektowany tak, aby automatycznie identyfikować model o najwyższej wartości wskaźnika R^2, a dla optymalnej architektury generować wykres punktowy porównujący wartości rzeczywiste z przewidywanymi, z naniesioną idealną prostą $y = x$.

Kompletny kod źródłowy realizujący opisany proces badawczy przedstawiono na poniższym listingu:

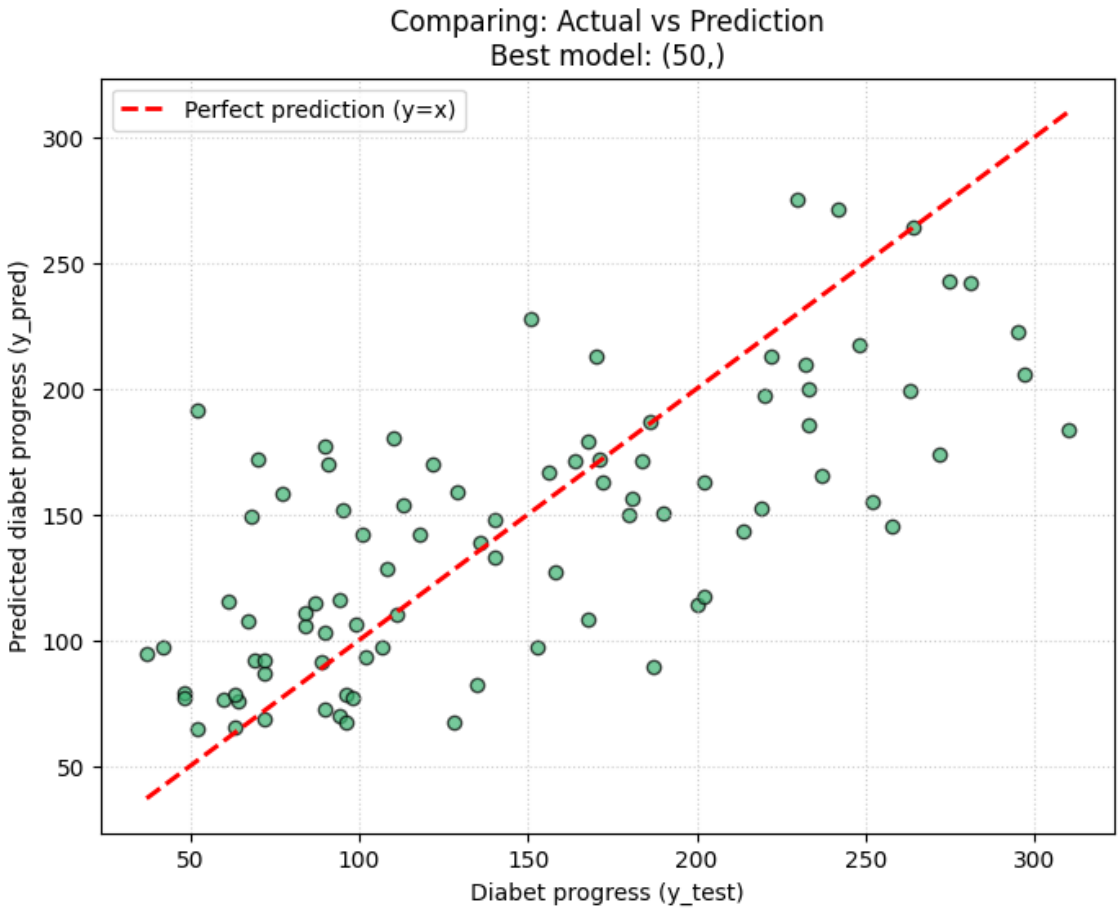
```
for arch in architectures:
    network = MLPRegressor(
        solver='adam',
        hidden_layer_sizes=arch,
        max_iter=5000,
        tol=0.001,
        random_state=42
    )

    network.fit(X_train_scaled, y_train)
    y_pred = network.predict(X_test_scaled)

    mse = mean_squared_error(y_test, y_pred)
    mape = mean_absolute_percentage_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    if r2 > best_r2:
        best_r2 = r2
        best_y_pred = y_pred
        best_arch = arch
```

3.2 Wyniki i wnioski

Po przeprowadzeniu procesu uczenia i ewaluacji pięciu różnych architektur sieci neuronowych na zbiorze *diabetes*, otrzymano wyniki zestawione w Tabeli 2. Do oceny wykorzystano błąd średniokwadratowy (MSE), średni absolutny błąd procentowy (MAPE) oraz współczynnik determinacji (R^2).

Architektura sieci	**MSE**	**MAPE**	R^2
(10)	2778.16	0.3847	0.4756
(50)	**2635.84**	**0.3492**	**0.5025**
(50, 50)	9788.55	0.6888	-0.8475
(100, 50)	6624.27	0.5635	-0.2503
(100, 100, 100)	7705.49	0.5867	-0.4544

Tabela 2: Zestawienie wyników ewaluacji modeli dla zbioru diabetes



Na podstawie uzyskanych danych oraz wygenerowanego wykresu predykcji można wyciągnąć następujące wnioski:

1. **Zjawisko przeuczenia (Overfitting):** Zbiór danych *diabetes* jest stosunkowo mały (442 próbki). Zastosowanie głębokich architektur, takich jak `(50, 50)`, `(100, 50)` czy `(100, 100, 100)`, doprowadziło do drastycznego spadku jakości modelu. Uzyskały one ujemne wartości współczynnika R^2, co w praktyce oznacza, że modele te radzą sobie gorzej niż funkcja zwracająca stałą wartość średnią. Złożone sieci "nauczyły się"szumu w danych treningowych, tracąc zdolność generalizacji.

2. **Analiza najlepszego modelu i wykresu:** Najlepsze rezultaty osiągnęła jednowarstwowa sieć o strukturze `(50)`. Zapewniła ona najwyższy współczynnik R^2 (0.5025) oraz najmniejsze błędy MSE i MAPE. Wygenerowany dla tego modelu wykres rozrzutu (ang. *scatter plot*) pokazuje relację między wartościami rzeczywistymi a przewidywanymi. Choć punkty nie układają się idealnie na prostej $y = x$, to jednak wyraźnie grupują się wokół niej, podążając za jej trendem. Biorąc pod uwagę dużą wariancję w danych medycznych, taki rozrzut jest zjawiskiem naturalnym. Warto również odnotować, że dla najprostszych modeli algorytm optymalizacyjny zgłosił *ConvergenceWarning* (brak pełnej zbieżności po 5000 epok), jednak nie przeszkodziło to w osiągnięciu najlepszych wyników.

3. **Wybór najmniejszej zadowalającej sieci:** Zgodnie z celem zadania, poszukiwano najmniejszej architektury zapewniającej satysfakcjonujące wyniki.

Choć model `(50,)` jest formalnie najlepszy, różnica w jakości między nim a skrajnie prostym modelem `(10,)` jest relatywnie niewielka (R^2 na poziomie 0.47 vs 0.50). W związku z tym, za **najmniejszą sieć o zadowalających wynikach można uznać architekturę** `(10,)`. Oferuje ona optymalny kompromis między bardzo niską złożonością obliczeniową a akceptowalną zdolnością do aproksymacji postępu choroby.

Eksperyment jednoznacznie dowodzi, że w przypadku niewielkich i zaszumionych medycznych zbiorów danych, proste i płytkie modele sieci neuronowych są znacznie bezpieczniejszym i skuteczniejszym wyborem niż głębokie architektury.

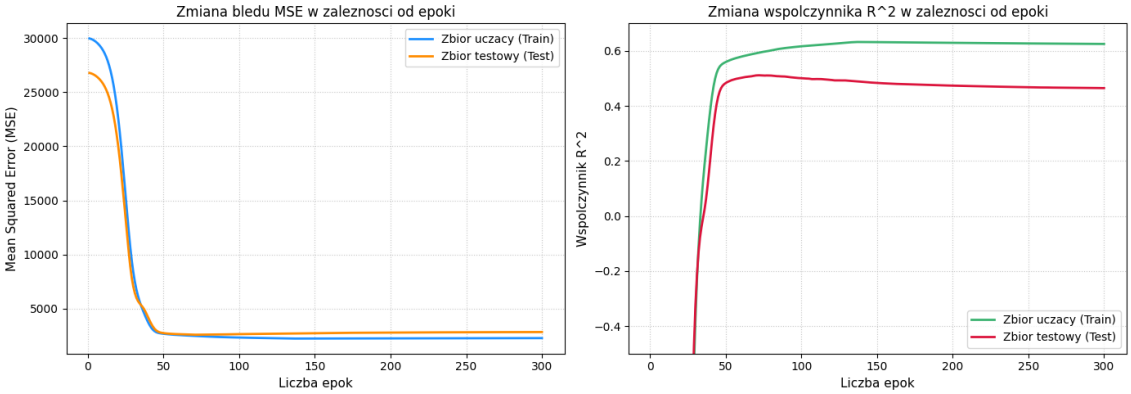
4 Zadanie 3

Celem trzeciego zadania jest szczegółowa analiza dynamiki procesu uczenia wybranej, optymalnej architektury sieci neuronowej. Istota pracy polega na wygenerowaniu i interpretacji krzywych uczenia (ang. *learning curves*), obrazujących zależność dwóch wybranych metryk ewaluacyjnych od liczby zrealizowanych epok treningowych. Pozwala to na wizualną ocenę zbieżności algorytmu optymalizacyjnego, weryfikację stabilności modelu oraz diagnozę potencjalnych zjawisk przeuczenia (ang. *overfitting*) lub niedouczenia (ang. *underfitting*). Uzyskane wyniki poddano analizie w kontekście teoretycznych zagadnień z zakresu uczenia maszynowego omawianych podczas zajęć.

4.1 Realizacja zadania

W celu dogłębnej analizy procesu treningowego, dla wyselekcjonowanej wcześniej optymalnej architektury sieci wygenerowano krzywe uczenia (ang. *learning curves*). Ewaluację zrealizowano poprzez iteracyjne douczanie modelu w pętli obejmującej 300 epok, co osiągnięto dzięki zastosowaniu parametru `warm_start=True` oraz sekwencyjnemu zwiększaniu limitu iteracji. Po każdym kroku algorytmu optymalizacyjnego wyznaczano i rejestrowano wartości błędu średniokwadratowego (MSE) oraz współczynnika determinacji (R^2) niezależnie dla podzbioru uczącego i testowego. Zgromadzone w ten sposób dane posłużyły do wygenerowania wykresów liniowych, pozwalających na wizualną ocenę dynamiki zbieżności modelu oraz weryfikację jego zdolności do generalizacji.

4.2 Wyniki i wnioski



Rysunek 1: krzywe uczenia

Na podstawie wygenerowanych wykresów krzywych uczenia (błędu MSE oraz współczynnika R^2) można sformułować następujące wnioski, ściśle nawiązujące do teorii uczenia maszynowego:

1. **Szybka zbieżność w początkowej fazie:** W początkowym etapie treningu (mniej więcej do 50. epoki) obserwujemy drastyczny spadek błędu MSE oraz gwałtowny wzrost współczynnika determinacji R^2 dla obu zbiorów. Świadczy to o tym, że sieć skutecznie i szybko wychwytuje główne, nieliniowe wzorce w danych medycznych pacjentów, a algorytm optymalizacyjny prawidłowo i dynamicznie aktualizuje wagi.

2. **Punkt optymalnej generalizacji (Optimum):** Analizując krzywe dla zbioru testowego (pomarańczowa dla MSE, czerwona dla R^2), można precyzyjnie wskazać moment najwyższej skuteczności modelu. Wypada on w okolicach 70-80. epoki. Współczynnik R^2 osiąga w tym punkcie swoje maksimum (ok. 0.5), a błąd MSE przyjmuje wartość minimalną. Jest to moment, w którym sieć ma najlepszą zdolność uogólniania wiedzy na nowe, niewidziane wcześniej dane.

3. **Zjawisko przeuczenia (Overfitting/Overtraining):** Powyżej 80. epoki wykresy zaczynają się rozchodzić, tworząc tzw. lukę generalizacyjną (ang. *generalization gap*). Krzywe dla zbioru uczącego wciąż powoli poprawiają swoje wartości (błąd maleje, R^2 rośnie), podczas gdy metryki dla zbioru testowego ulegają stopniowej degradacji (czerwona krzywa R^2 powoli, lecz zauważalnie opada). Oznacza to, że w późniejszych epokach sieć zaczyna "uczyć się na pamięć"specyficznego szumu i odstępstw obecnych w zbiorze treningowym, kosztem trafności predykcji.